Cryptography and Network Security

11-3 MESSAGE AUTHENTICATION

A message digest does not authenticate the sender of the message. To provide message authentication, Alice needs to provide proof that it is Alice sending the message and not an impostor. The digest created by a cryptographic hash function is normally called a modification detection code (MDC). What we need for message authentication is a message authentication code (MAC).

Topics discussed in this section:

11.3.1 Modification Detection Code (MDC)

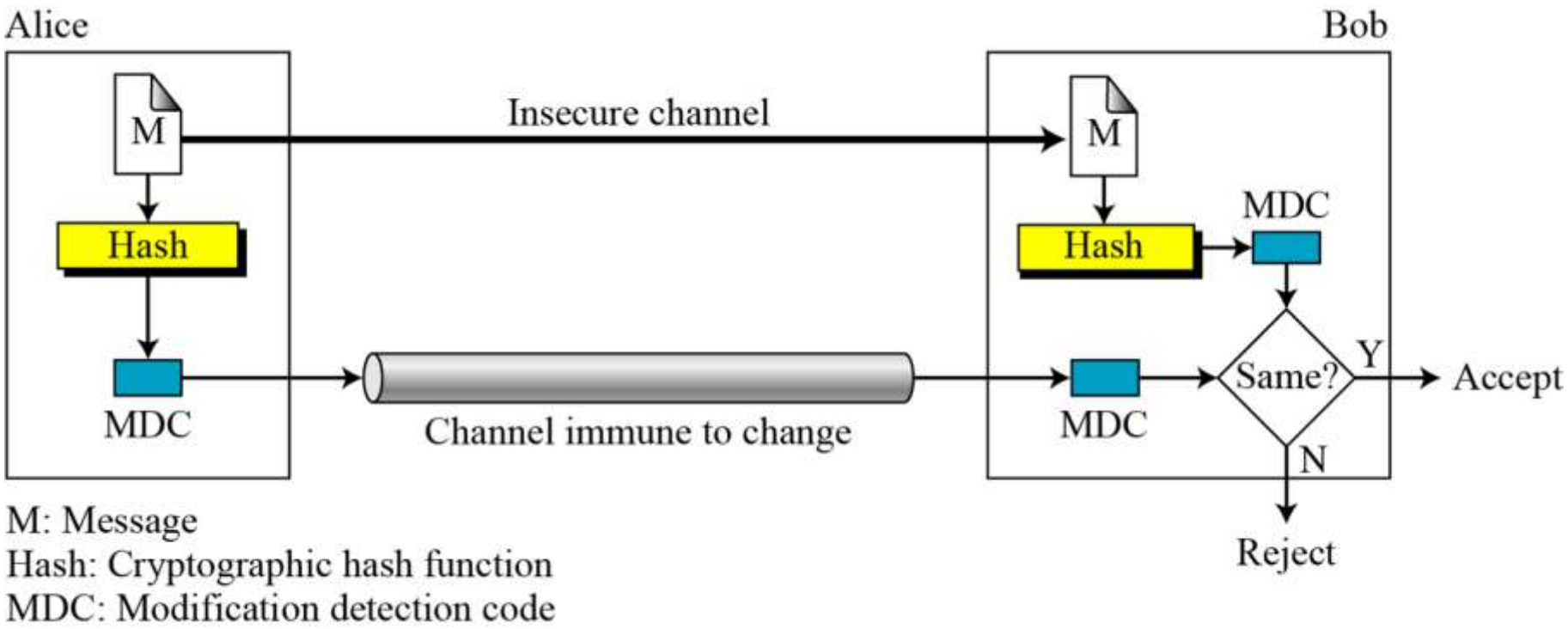
11.3.2 Message Authentication Code (MAC)

11.3.1 Modification Detection Code (MDC)

A modification detection code (MDC) is a message digest that can prove the integrity of the message: that message has not been changed. If Alice needs to send a message to Bob and be sure that the message will not change during transmission, Alice can create a message digest, MDC, and send both the message and the MDC to Bob. Bob can create a new MDC from the message and compare the received MDC and the new MDC. If they are the same, the message has not been changed.

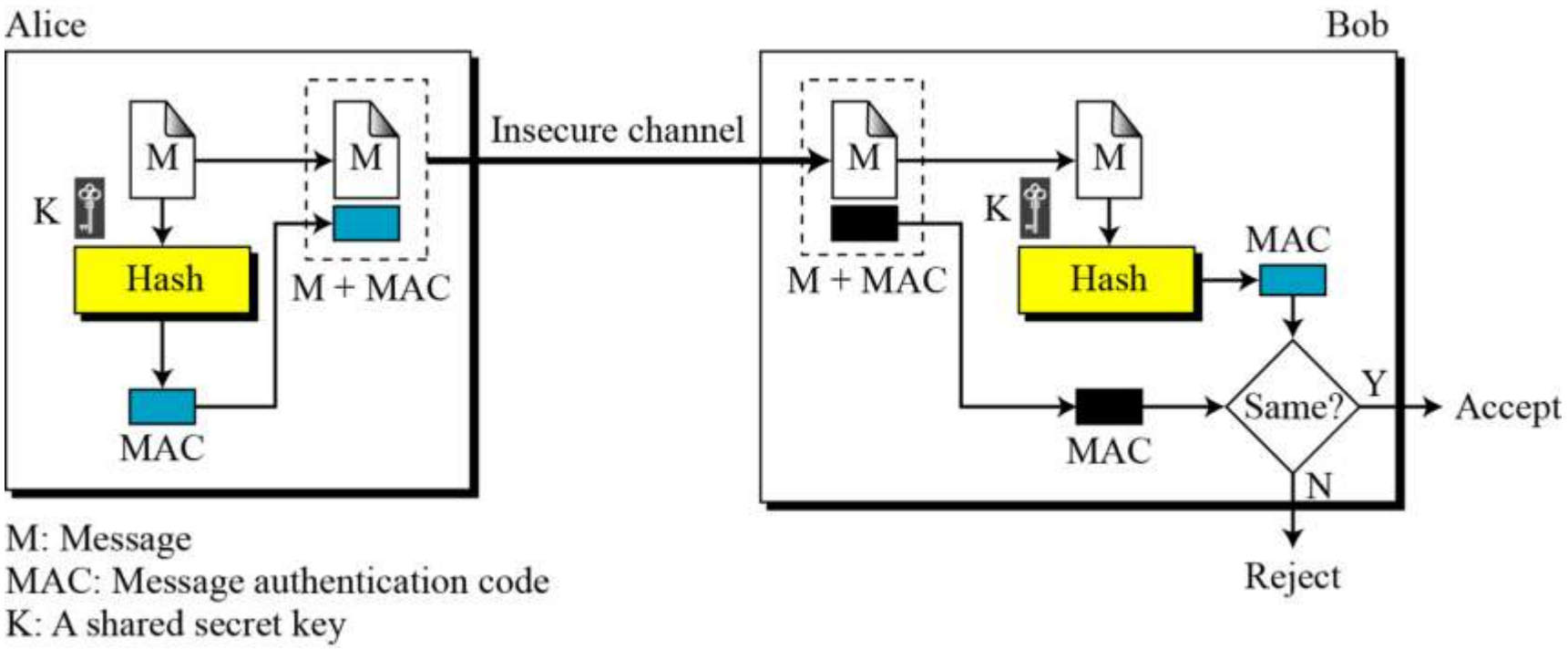
11.3.1 Continued

Figure 11.9 *Modification detection code (MDC)*



11.3.2 Message Authentication Code (MAC)

Figure 11.10 *Message authentication code*



11.3.2 Continued

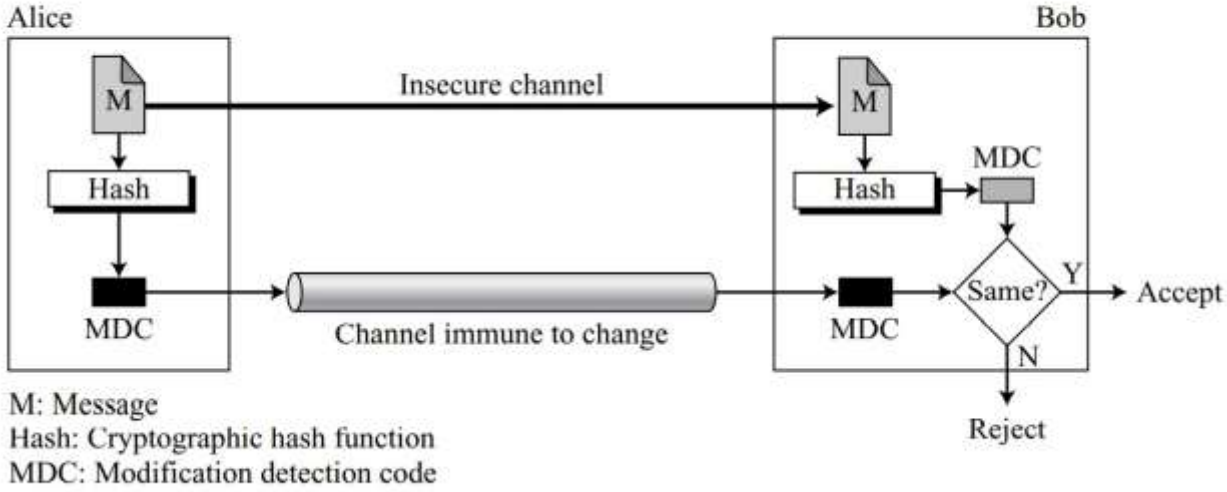
Note

The security of a MAC depends on the security of the underlying hash algorithm.

MESSAGE AUTHENTICATION - MDC

- MESSAGE AUTHENTICATION
  - Proof of source cannot be obtained through Message Digest
  - MD helps in knowing the correctness of the data not authenticate
- Modification Detection Code
  - Message Digest can only detect modifications in the message
  - If both Message and MDC are sent through the insecure channel, then Eve can change both and send

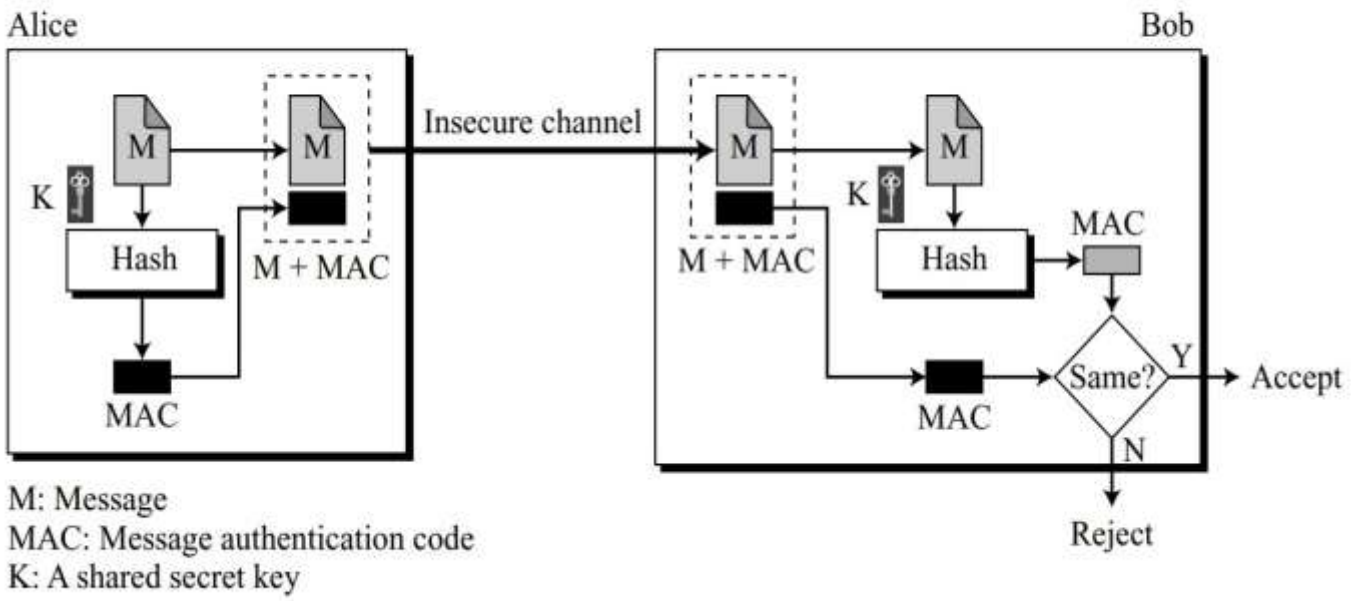
Figure 11.9 *Modification detection code (MDC)*



MESSAGE AUTHENTICATION-MAC

- Message Authentication Code (MAC)
  - Ensures integrity and authentication both
  - Secret key
  - Prefix MAC -K|M – concatenation done and sent to hash function

Figure 11.10 *Message authentication code*

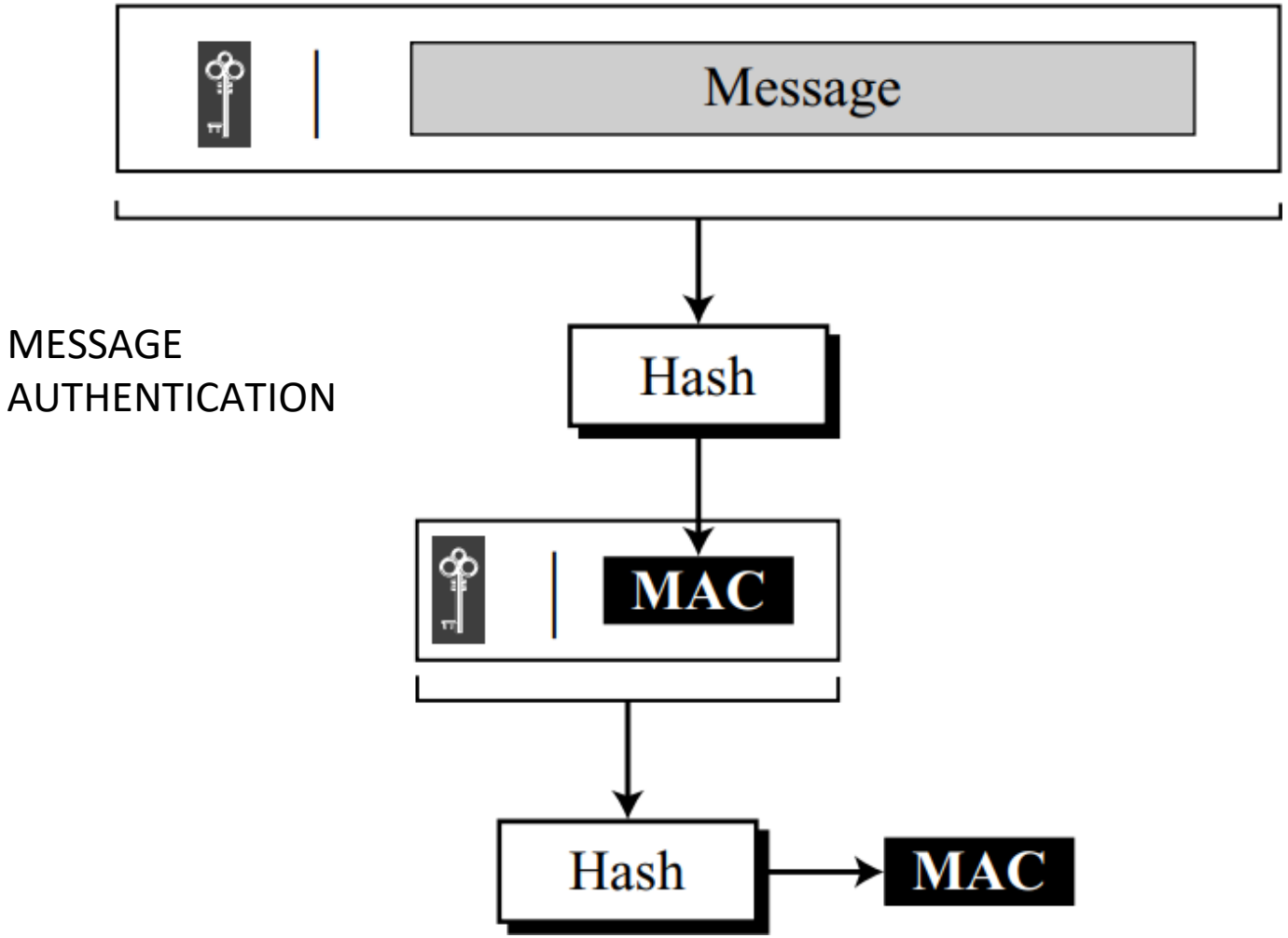


MESSAGE AUTHENTICATION

- Security of a MAC
  - Suppose Eve has intercepted the message M and the digest h(K|M).
  - There are three possible cases for forging:
  - 1. If the size of the key allows exhaustive search, Eve may prepend all possible keys at the beginning of the message and make a digest of the (K|M) to find the digest
  - 2. The size of the key is normally very large in a MAC, but Eve can use another tool: the preimage attack. She uses the algorithm until she finds X such that h(X) is equal to the MAC she has intercepted.
  - 3. Given some pairs of messages and their MACs, Eve can manipulate them to comeup with a new message and its MAC

MESSAGE AUTHENTICATION

- Nested MAC



MESSAGE AUTHENTICATION

HMAC: Hashed MAC

1. The message is divided into N blocks, each of b bits.
2. The secret key is left-padded with 0's to create a b-bit key. Note that it is recommended that the secret key (before padding) be longer than n bits, where n is the size of the HMAC.
3. The result of step 2 is exclusive-ored with a constant called ipad (input pad) to create a b-bit block. The value of ipad is the b/8 repetition of the sequence 00110110 (36 in hexadecimal).
4. The resulting block is prepended to the N-block message. The result is N + 1 blocks.
5. The result of step 4 is hashed to create an n-bit digest. We call the digest the intermediate HMAC.

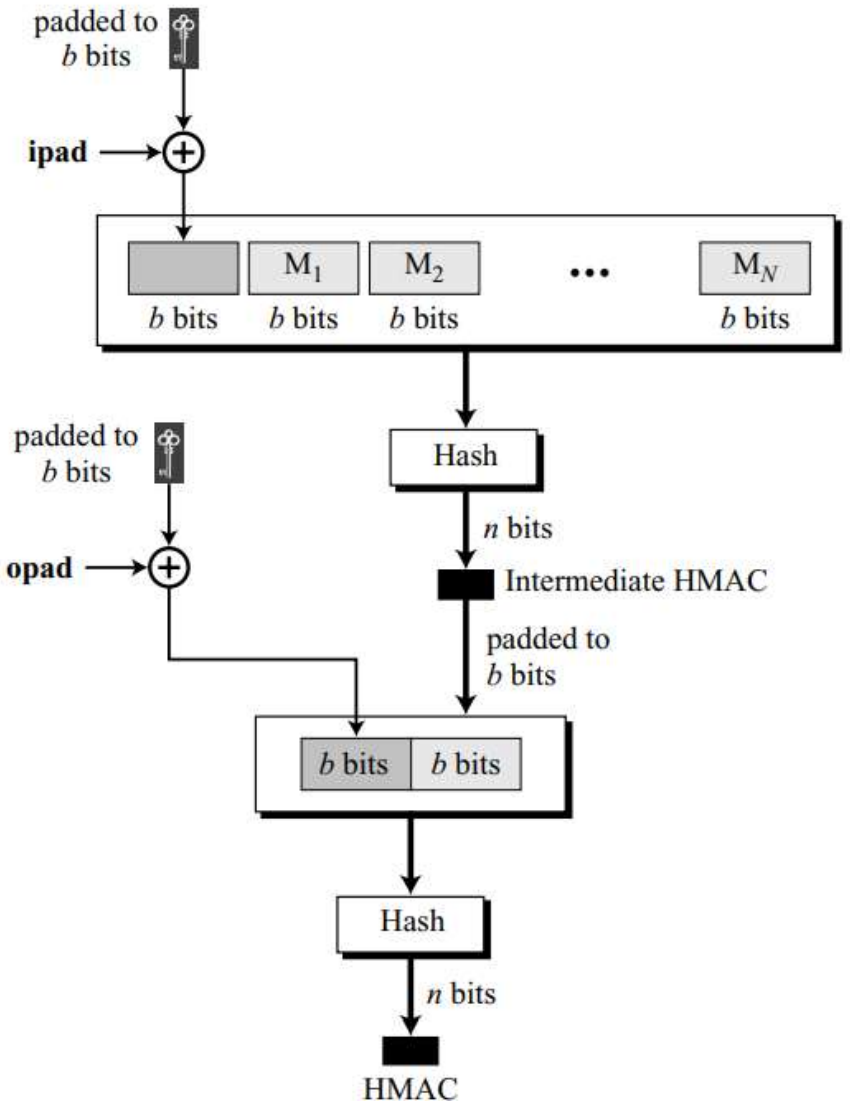
MESSAGE AUTHENTICATION

6. The intermediate n-bit HMAC is left padded with 0s to make a b-bit block.
7. Steps 2 and 3 are repeated by a different constant opad (output pad). The value of opad is the b/8 repetition of the sequence 01011100 (5C in hexadecimal).
8. The result of step 7 is prepended to the block of step 6.
9. The result of step 8 is hashed with the same hashing algorithm to create the final n-bit HMAC.

HMAC

NIST has issued a standard (FIPS 198) for a nested MAC that is often referred to as HMAC

Figure 11.12 *Details of HMAC*



CMAC

CMAC- Cipher Block Chaining MAC

NIST has also defined a standard (FIPS 113) called Data Authentication Algorithm, or CMAC, or CBCMAC

CBC – Used for confidentiality every block output influences CT.

Here they are used only with next block

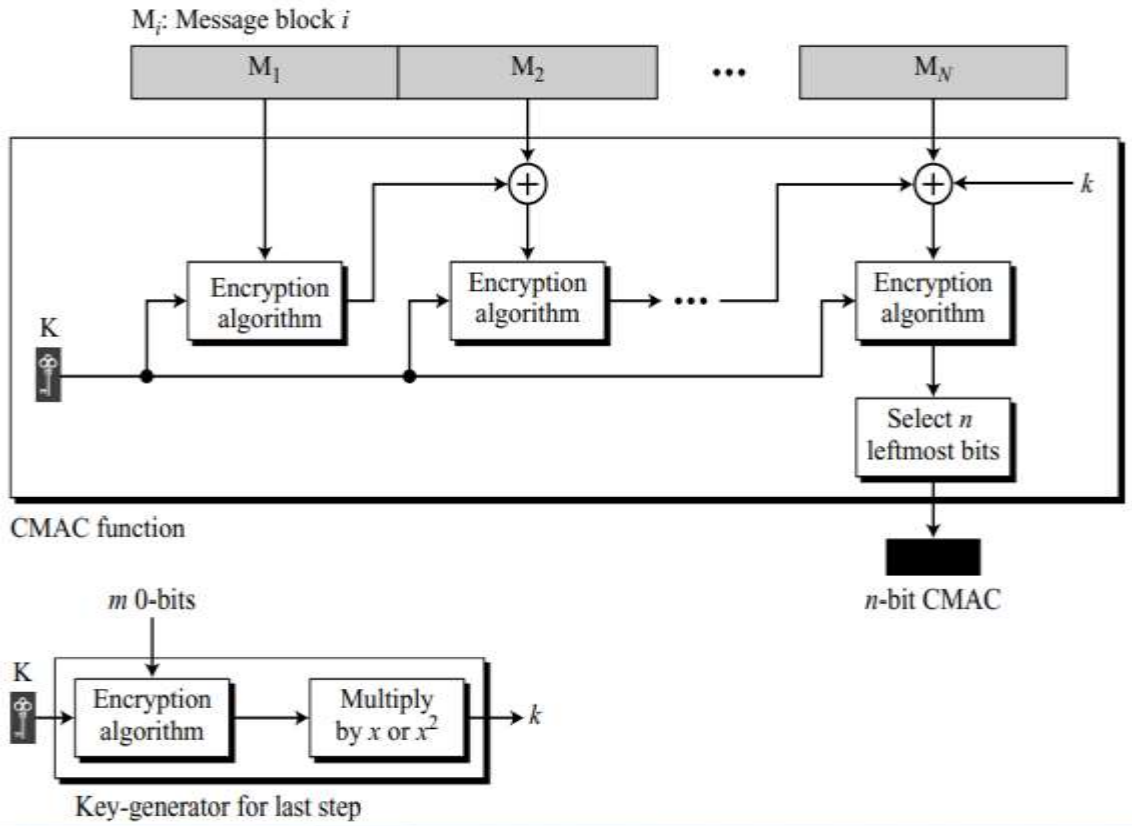
idea is to create one block of MAC from N blocks of plaintext using a symmetric-key cipher N times

Procedure followed as shown

If padding is applied to last block, x^2 is used for multiplication, if there is no padding, x is used.

Multiplication is in GF(2^m) with irreducible polynomial of degree m

Figure 11.13 *CMAC*



Cryptographic Hash Functions

- Digital Signature
  - Signature is for authenticity- Ex. A signature in a bank cheque
  - Electronic signature done by the sender as a proof to the recipient that the document comes from correct entity (proof of source)

- COMPARISON
  - Inclusion
    - CS: Cheque and signature are together
    - DS: document / message and signature are separate
  - Verification Method:

    Method of verifying the signature
    - CS: once a document is received, signature is verified with the signature in file
    - DS: a verification algorithm is used to verify a signature and the signature is not stored anywhere
  - Relationship:
    - CS: one to many relation -same signatures in many documents
    - Digital sign: one to one – each message has a different signature
  - Duplicity:
    - CS: sign in copy of the signed document and the original may be slightly different
    - DS: no change among the two unless a timestamp is used

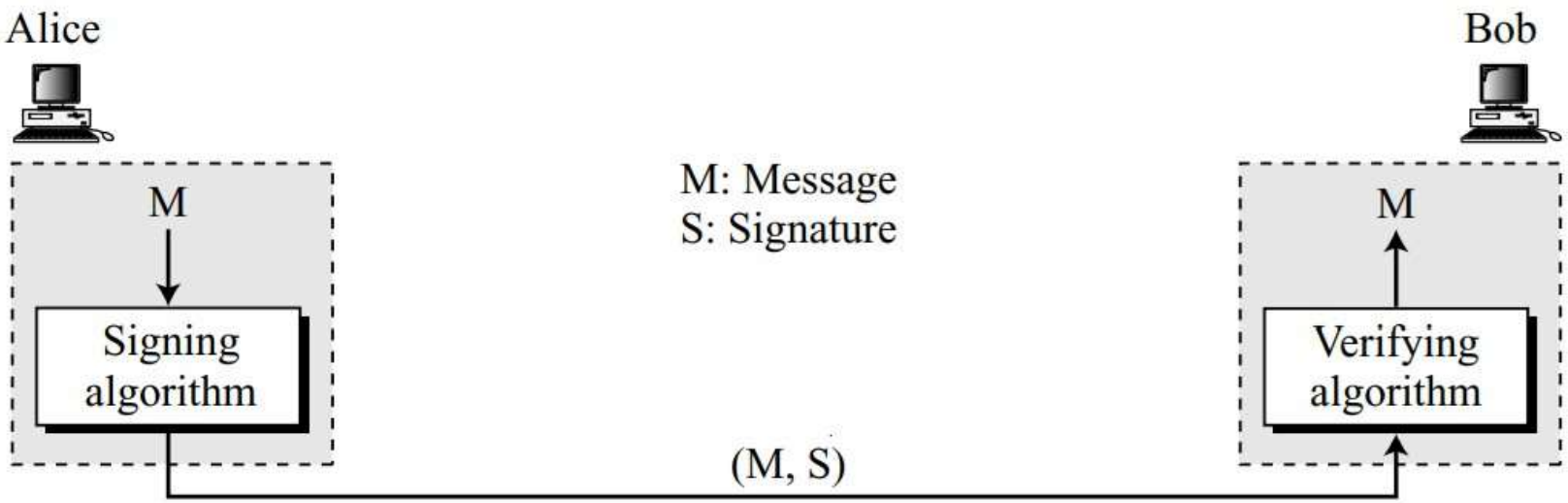
Difference between conventional and digital signature

Conventional signature	Digital signature
A document consists of conventional signature in it. A conventional signature is done on the document which needs to be authenticated and hence always forms a part of the document.	A digital signature is done on a different document than which is to be authenticated. The receiver receives digital signature and the message or document and if the signature belongs to the right person, it accepts the message otherwise rejects it.
It is a one-to-many relationship since one person uses same signature to sign various documents.	It is a one-to-one relationship since every time communication takes place, the signature is changed as well.
The verification of this type of signature is done by simply comparing the signature on the document with the signature stored in the file.	A digital signature is verified by applying verification technique on the message and the signature.
A conventional signature can be easily identified if it is different from the file on which the signature is stored.	In this case, if there is no timestamp on the document then the signature can be duplicated easily.
It is exclusively linked to an individual.	It depends on the verification provided by the signer and the Certification Authority.

Cryptographic Hash Functions

PROCESS

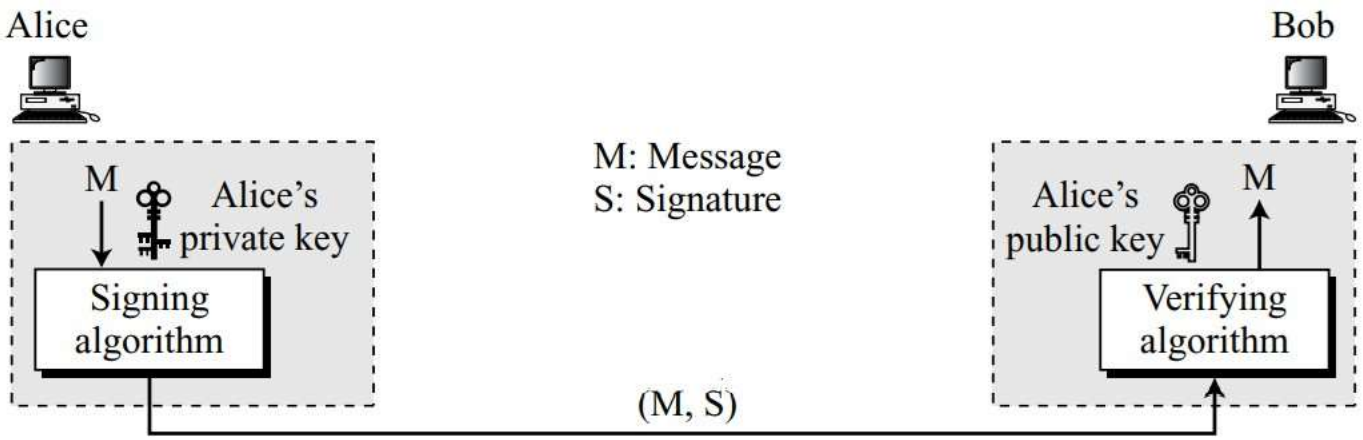
Figure 13.1 *Digital signature process*



Cryptographic Hash Functions

- Need for Keys
  - CS is like private key which is available only with the owner of document
  - Public key is available in a file and anyone can use it to verify
  - Asymm key crypto - but keys used are completely diff from regular procedure
  - One single key cannot be used – ends in forgery
  - For every session, a new pair is required

Figure 13.2 *Adding key to the digital signature process*

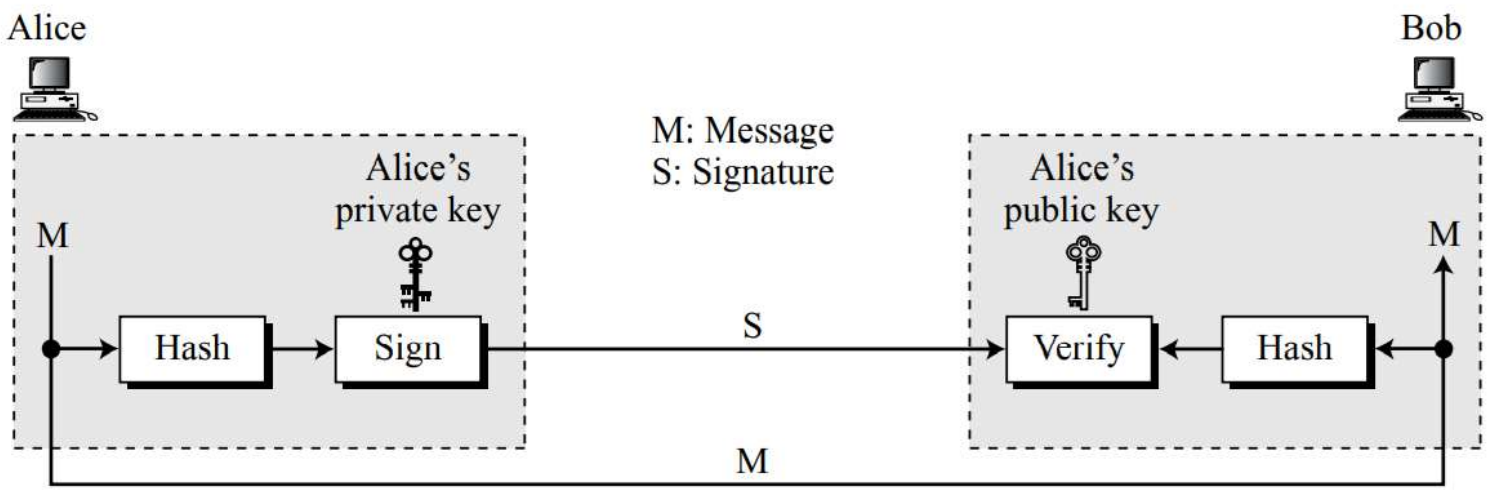


- Summary:
- A digital signature needs a public-key system. The signer signs with her private key; the verifier verifies with the signer’s public key.
- A cryptosystem uses the private and public keys of the receiver: a digital signature uses the private and public keys of the sender.

Cryptographic Hash Functions

- Signing the Digest
  - Asymm key cryp not for long messages
  - In DS sys, messages are generally long
  - Soln: sign a digest of the message(shorter)

Figure 13.3 *Signing the digest*



Cryptographic Hash Functions

ATTACKS ON DIGITAL SIGNATURE

- Attack Types
  - Key-Only Attack
    - Eve has access to only public key
    - To forge, Eve needs to create Alice's signature
    - Like ciphertext-only attack
  - Known-Message Attack
    - Eve has access to some M,S Pairs
    - With this tries to create a message and tries to sign
    - Like known- PT attack
  - Chosen-Message Attack
    - Eve somehow makes Alice sign one or more messages
    - Eve now has a chosen message/ signature pair
    - Chosen plaintext attack

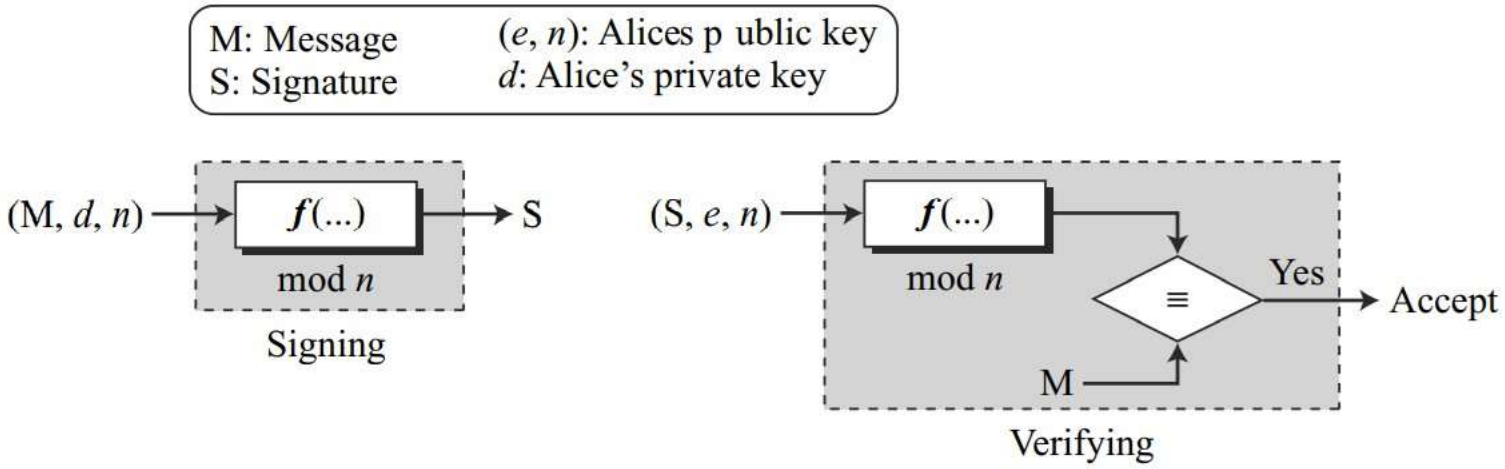
Cryptographic Hash Functions

- Forgery Types
  - Existential Forgery
    - Eve may create a valid M,S pair but not the one she can use
    - These could be syntactically or semantically unintelligible
  - Selective Forgery
    - Eve may be able to forge Alice Signature on a message with the content selectively
      chosen by eve
    - Probability is very low, but not negligible

Cryptographic Hash Functions

- DIGITAL SIGNATURE SCHEMES
  - RSA Digital Signature Scheme

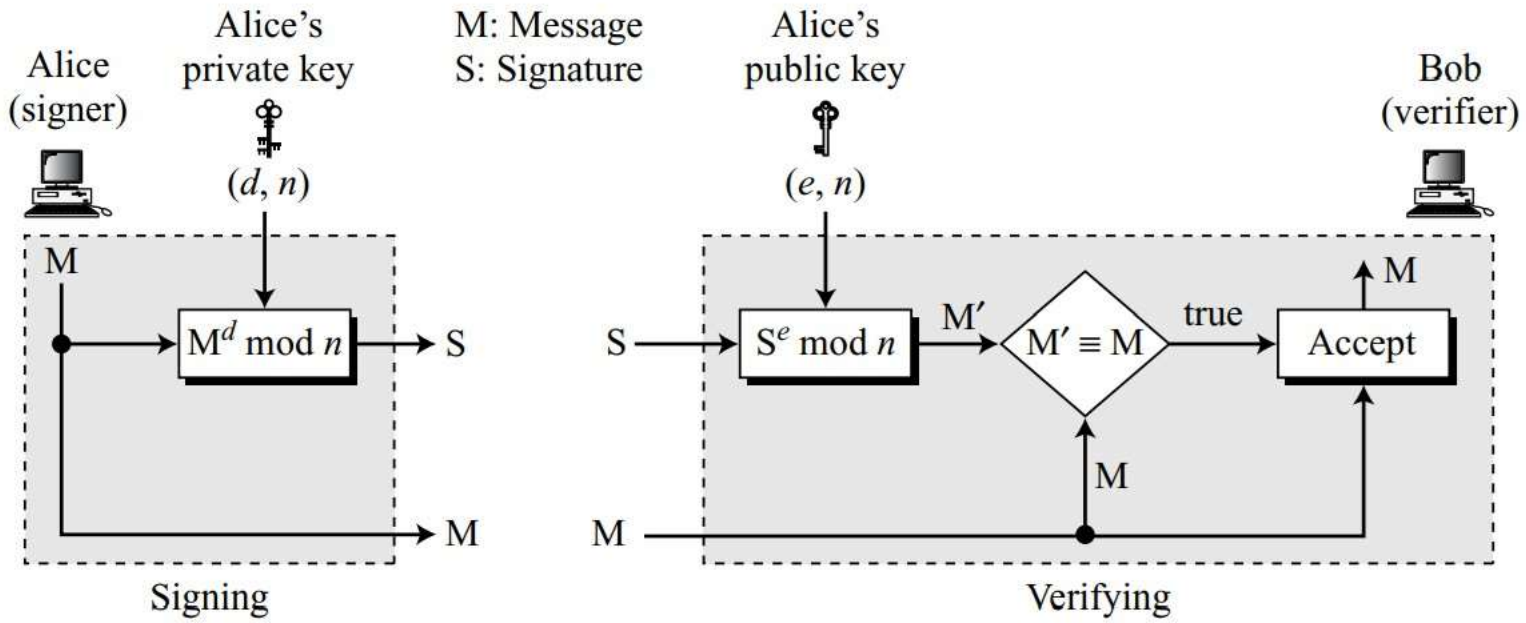
Figure 13.6 *General idea behind the RSA digital signature scheme*



Cryptographic Hash Functions

- Key Generatic

Figure 13.7 *RSA digital signature scheme*



$$M' \equiv M \pmod{n} \quad \rightarrow \quad S^e \equiv M \pmod{n} \quad \rightarrow \quad M^{d \times e} \equiv M \pmod{n}$$

- Attacks on RSA Signature
  - Key-Only Attack
    - Has only Alice pub key
    - Creates another message- M'== S^e mod n
    - Not very easy to solve
    - Like discrete log attack
  - Known-Message Attack
    - Eve uses multiplicative property of RSA

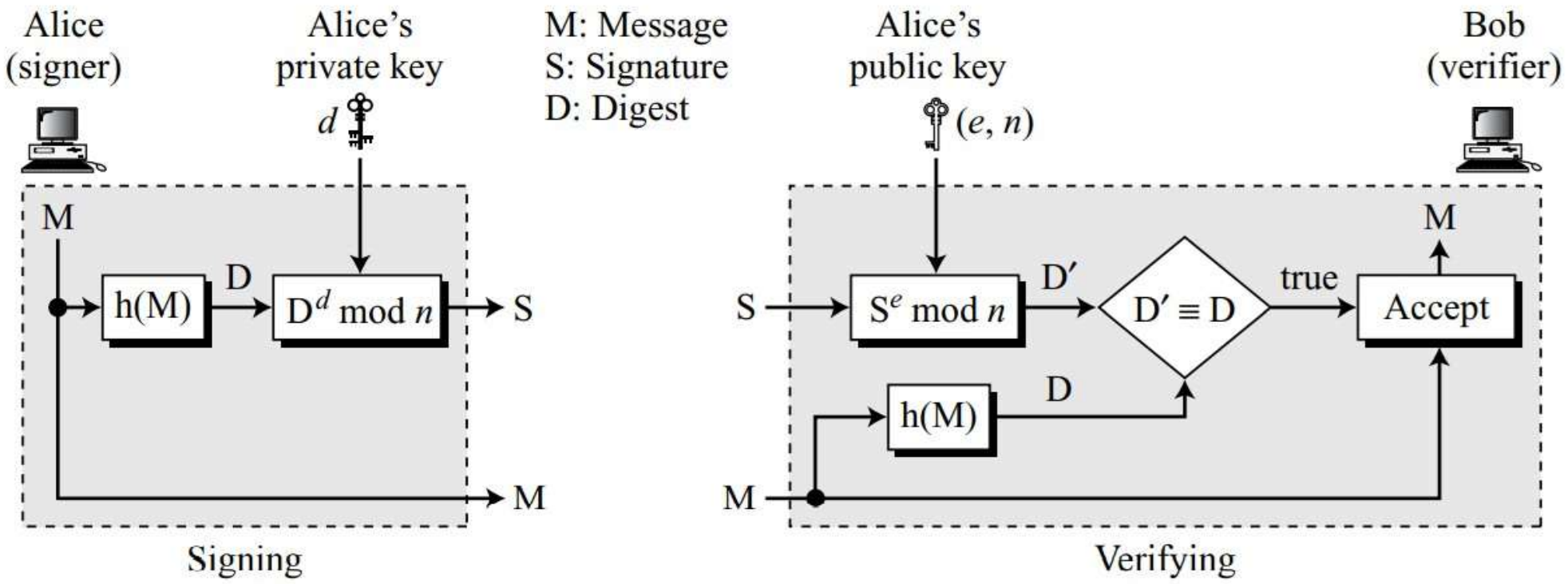
that Eve has intercepted two message-signature pairs (M_1, S_1) and (M_2, S_2) that have been created using the same private key. If $M = (M_1 \times M_2) \bmod n$, then $S = (S_1 \times S_2) \bmod n$. This is simple to prove because we have

$$S = (S_1 \times S_2) \bmod n = (M_1^d \times M_2^d) \bmod n = (M_1 \times M_2)^d \bmod n = M^d \bmod n$$

- Chosen Message Attack
  - Uses Multiplicative property
  - If Eve choses two messages and gets it signed from Alice
  - Very serious attack
  - Multiplicative attack

Cryptographic Hash Functions

Figure 13.8 *The RSA signature on the message digest*



Cryptographic Hash Functions

- Attacks on RSA Signed Digests
- Key-Only Attack

  A)
  - Intercepts (S,M)
  - Try to find M’ which gives the same hash h(M)=h(M’)
  - If hash algo is second preimage resistant, attack is diff

  B)
  - Finds M and M’ with same signature.
  - Makes Alice sign M- has two pairs with same sign
  - If hash algo is colli resis then this is difficult

  C)
  - Random D found which matches with random sign S. Finds Message D=h(M)
  - If preimage resis this is difficult

Cryptographic Hash Functions

- Known-Message Attack
  - Eve has two - (M1,S1) and (M2,S2) – same private key
  - Calculates S≡S1 XS2
  - IF can find an mess M such that h(M) ≡ h(M1) X h(M2) she had forged a new message
  - Finding M is difficult if HA is preimage resistant
- Chosen Known-Message Attack
  - Eve can ask Alice to sign two legitimate mess M1 and M2 for her
  - Eve creates a new signature S ≡ S1 X S2
  - Calculates h(M) ≡ h(M1)x h(M2)- if a message M can be found given h(M) the new message is forgery
  - Diff if HA is preimage resistant

Chapter 15

Key Management

Copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Objectives

- To explain the need for a key-distribution center
- To show how a KDC can create a session key
- To show how two parties can use a symmetric-key agreement protocol to create a session key
- To describe Kerberos as a KDC and an authentication protocol
- To explain the need for certification authorities for public keys
- To introduce the idea of a Public-Key Infrastructure (PKI) and explain some of its duties

15-1 SYMMETRIC-KEY DISTRIBUTION

Symmetric-key cryptography is more efficient than asymmetric-key cryptography for enciphering large messages. Symmetric-key cryptography, however, needs a shared secret key between two parties. The distribution of keys is another problem.

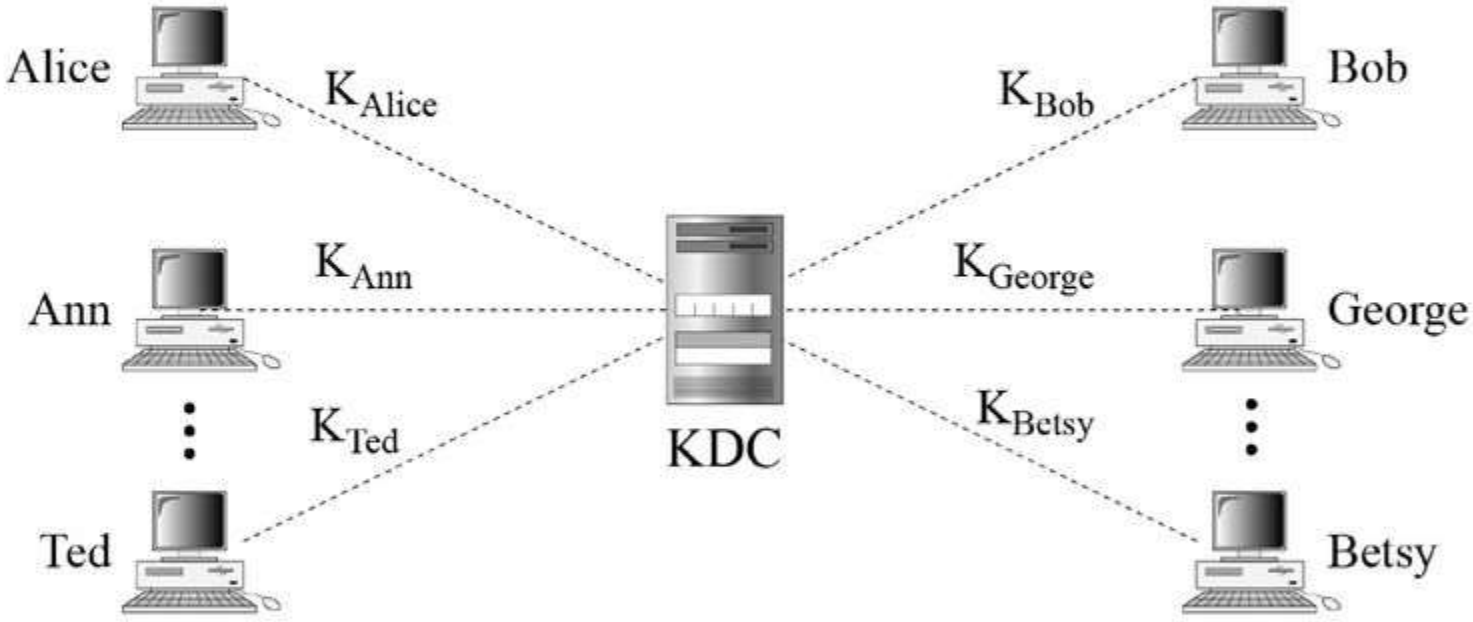
Topics discussed in this section:

15.1.1 Key-Distribution Center: KDC

15.1.2 Session Keys

15.1.1 Key-Distribution Center: KDC

Figure 15.1 *Key-distribution center (KDC)*



15.1.2 Session Keys

A KDC creates a secret key for each member. This secret key can be used only between the member and the KDC, not between two members.

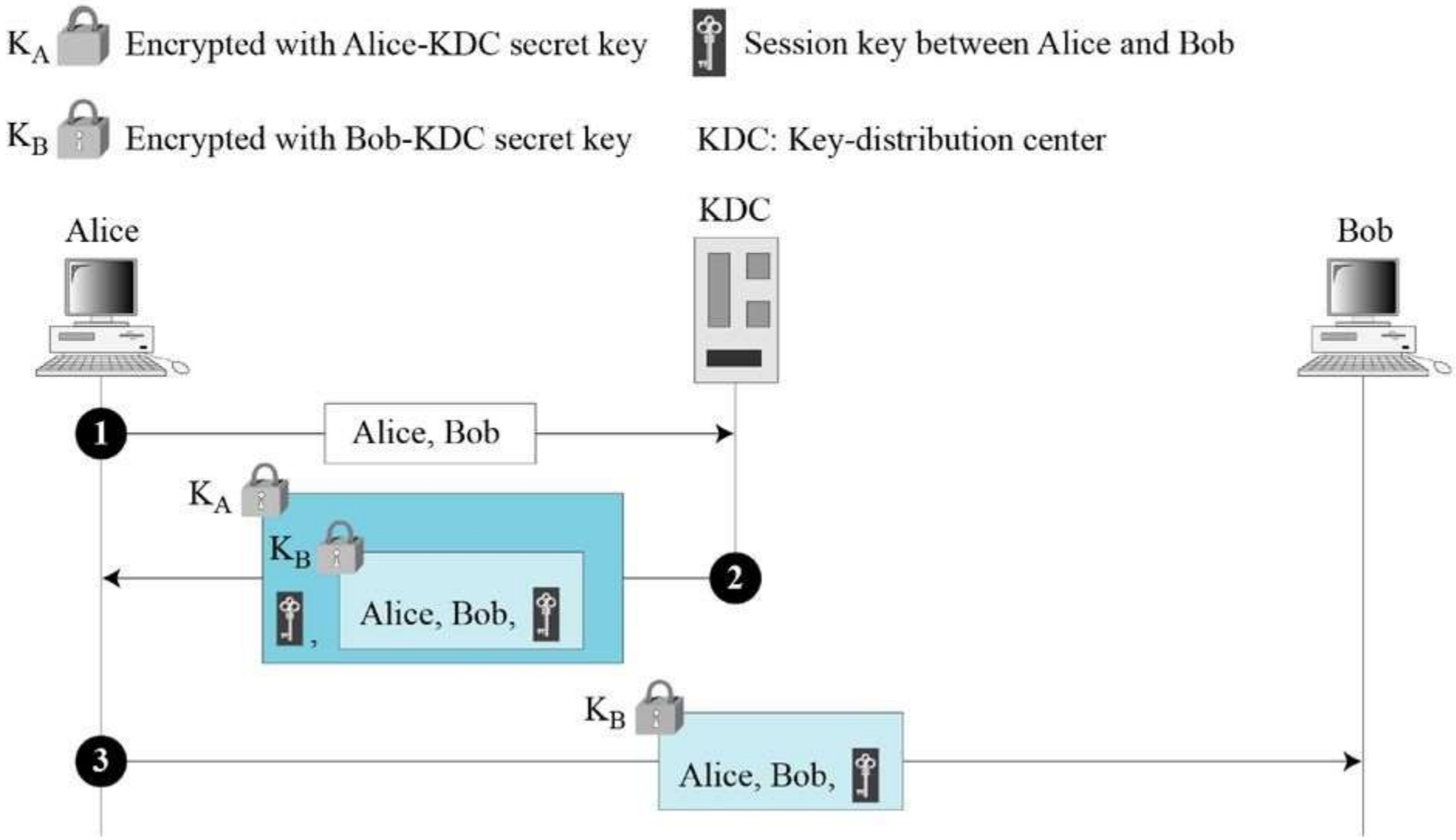
Note

A session symmetric key between two parties is used only once.

15.1.2 Continued

A Simple Protocol Using a KDC

Figure 15.4 *First approach using KDC*



15-2 KERBEROS

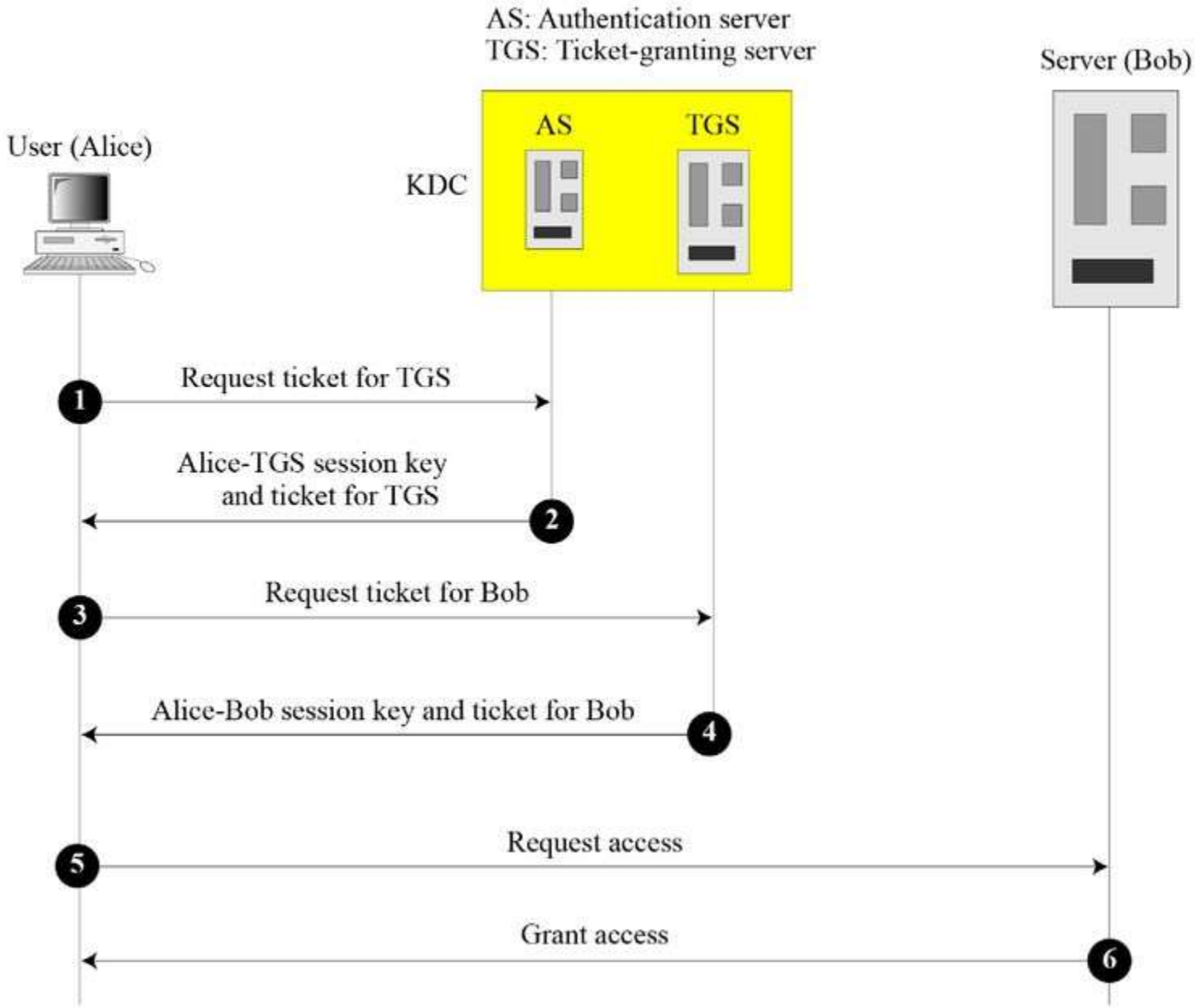
Kerberos is an authentication protocol, and at the same time a KDC, that has become very popular. Several systems, including Windows 2000, use Kerberos. Originally designed at MIT, it has gone through several versions.

Topics discussed in this section:

15.2.1 Servers
15.2.2 Operation
15.2.3 Using Different Servers
15.2.4 Kerberos Version 5
14.2.5 Realms

15.2.1 Servers

Figure 15.7 *Kerberos servers*



15.2.1 Continued

Authentication Server (AS)

The authentication server (AS) is the KDC in the Kerberos protocol.

Ticket-Granting Server (TGS)

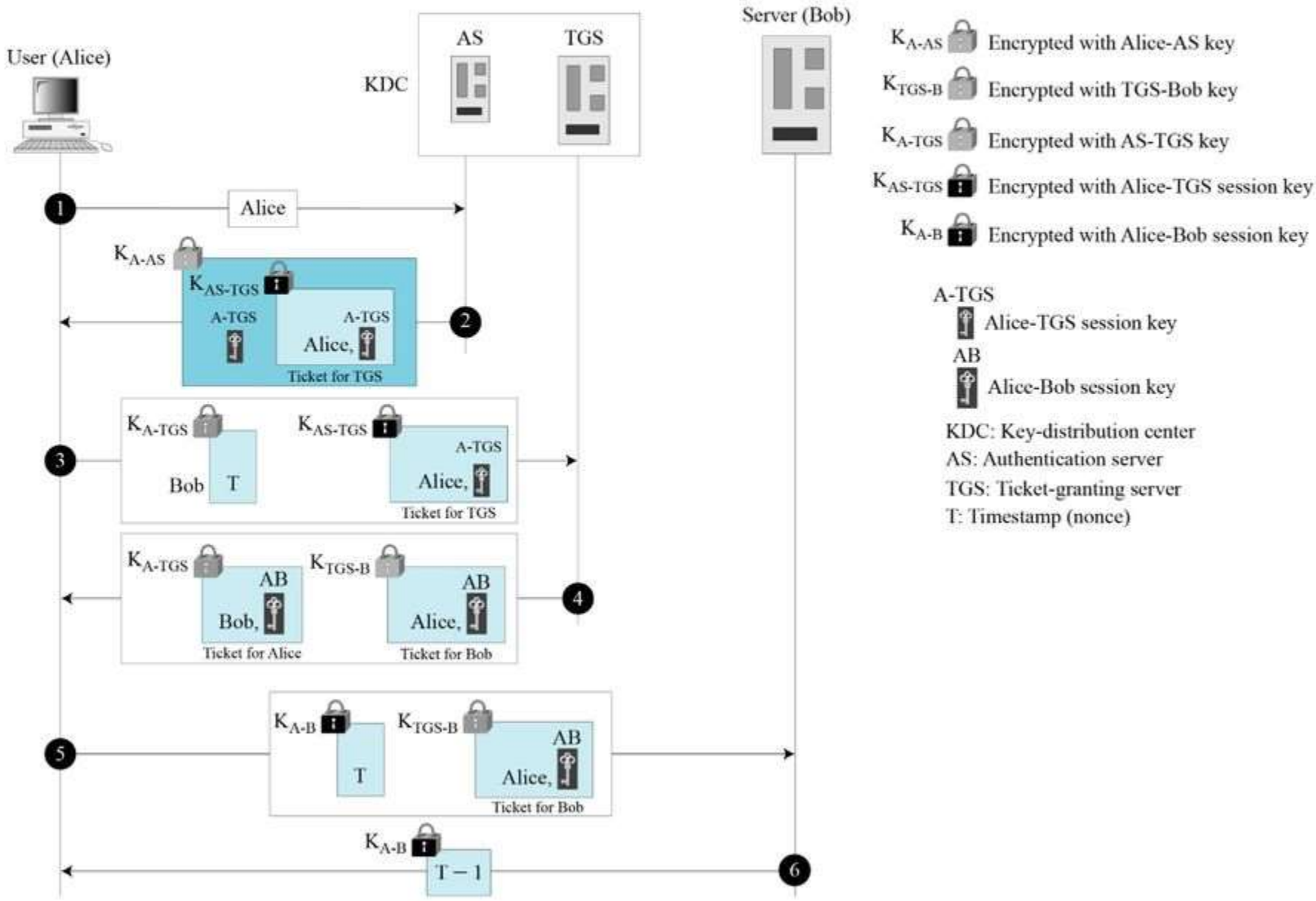
The ticket-granting server (TGS) issues a ticket for the real server (Bob).

Real Server

The real server (Bob) provides services for the user (Alice).

15.2.2 Operation

Figure 15.8 *Kerberos example*



15.2.3 Using Different Servers

Note that if Alice needs to receive services from different servers, she need repeat only the last four steps.

15.2.4 Kerberos Version 5

The minor differences between version 4 and version 5 are briefly listed below:

1) ***Version 5 has a longer ticket lifetime.***
2) ***Version 5 allows tickets to be renewed.***
3) ***Version 5 can accept any symmetric-key algorithm.***
4) ***Version 5 uses a different protocol for describing data types.***
5) ***Version 5 has more overhead than version 4.***

15.2.5 Realms

Kerberos allows the global distribution of ASs and TGSs, with each system called a realm. A user may get a ticket for a local server or a remote server.

15-3 SYMMETRIC-KEY AGREEMENT

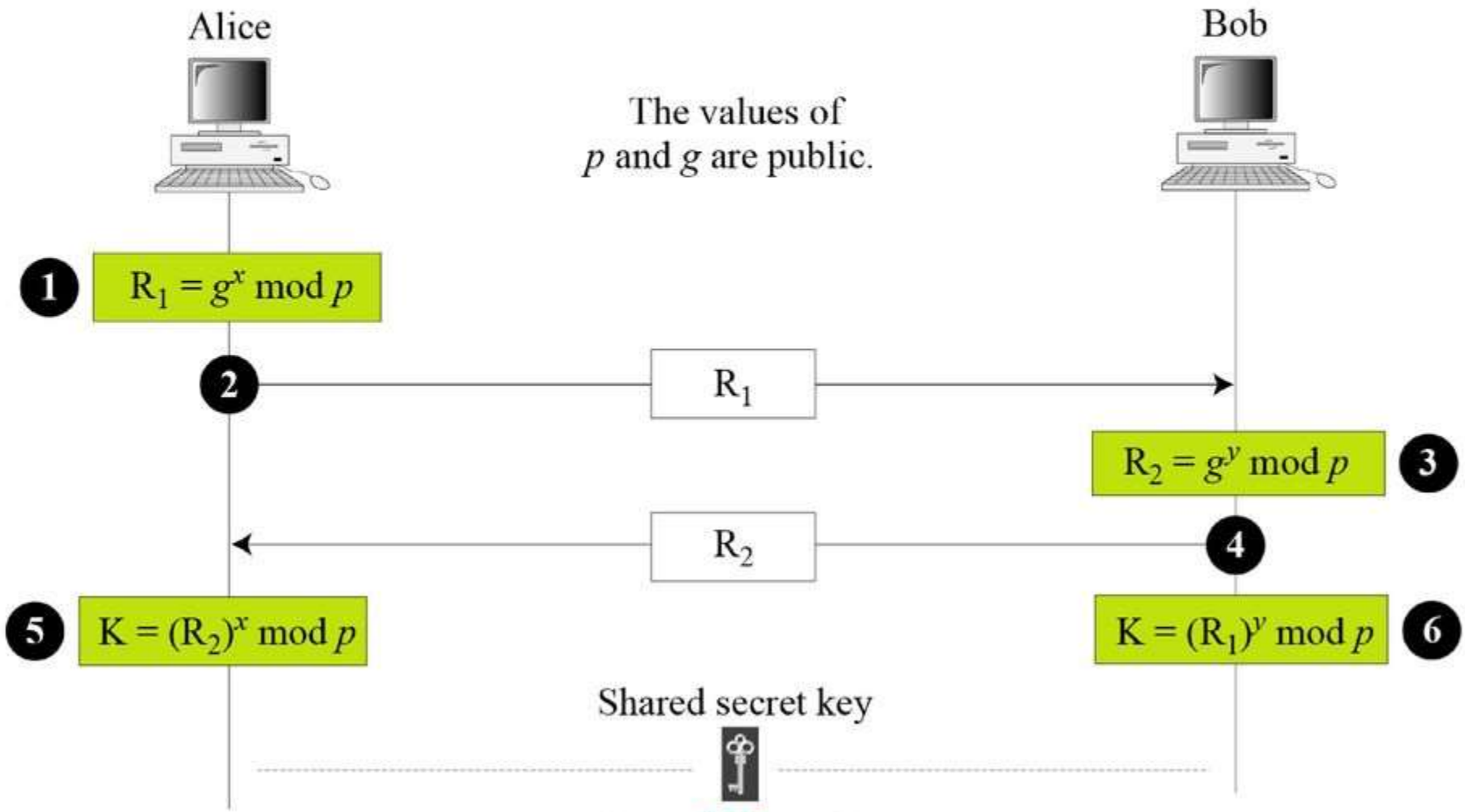
Alice and Bob can create a session key between themselves without using a KDC. This method of session-key creation is referred to as the symmetric-key agreement.

Topics discussed in this section:

15.3.1 Diffie-Hellman Key Agreement
15.3.2 Station-to-Station Key Agreement

15.3.1 Diffie-Hellman Key Agreement

Figure 15.9 *Diffie-Hellman method*



$$K = g^{xy} \bmod p$$

Note

The symmetric (shared) key in the Diffie-Hellman method is $K = g^{xy} \bmod p$.

Example 15.1

Let us give a trivial example to make the procedure clear. Our example uses small numbers, but note that in a real situation, the numbers are very large. Assume that $g = 7$ and $p = 23$. The steps are as follows:

1. **Alice chooses $x = 3$ and calculates $R_1 = 73 \bmod 23 = 21$.**
2. **Bob chooses $y = 6$ and calculates $R_2 = 76 \bmod 23 = 4$.**
3. **Alice sends the number 21 to Bob.**
4. **Bob sends the number 4 to Alice.**
5. **Alice calculates the symmetric key $K = 43 \bmod 23 = 18$.**
6. **Bob calculates the symmetric key $K = 216 \bmod 23 = 18$.**
7. **The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 718 \bmod 35 = 18$.**

15.3.1 Continued

Example 15.2

Let us give a more realistic example. We used a program to create a random integer of 512 bits (the ideal is 1024 bits). The integer *p* is a 159-digit number. We also choose *g*, *x*, and *y* as shown below:

p	764624298563493572182493765955030507476338096726949748923573772860925 2356666607554236374233096611800333381061947301309504147387009991780436548785807987581
g	2
x	557
y	273

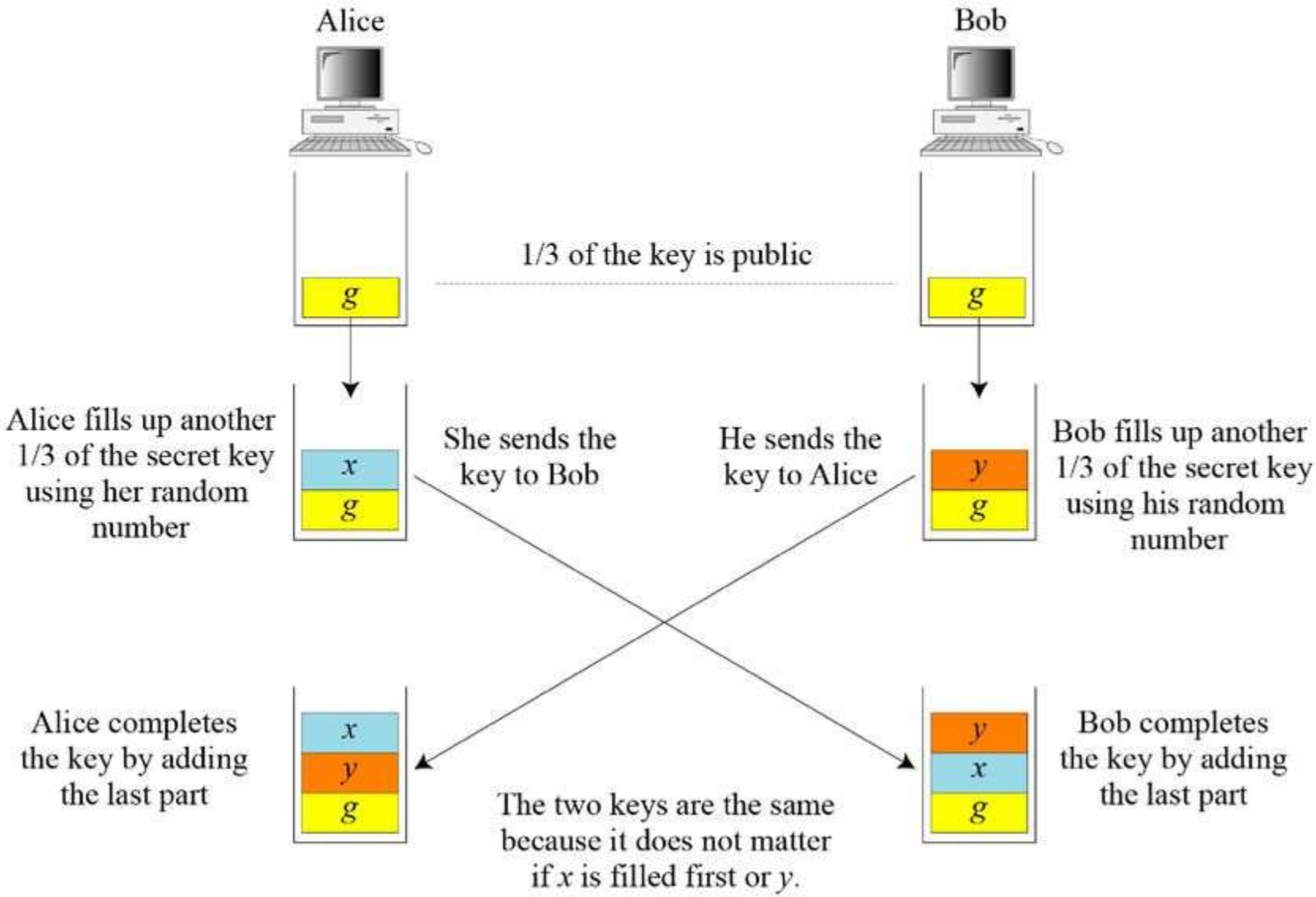
15.3.1 Continued

Example 15.2 *Continued*

The following shows the values of R_1, R_2, and K.

R_1	844920284205665505216172947491035094143433698520012660862863631067673 619959280828586700802131859290945140217500319973312945836083821943065 966020157955354
R_2	435262838709200379470747114895581627636389116262115557975123379218566 310011435718208390040181876486841753831165342691630263421106721508589 625520128859414 3
K	155638000664522290596225827523270765273218046944423678520320400146406 500887936651204257426776608327911017153038674561252213151610976584200 1204086433617740

Figure 15.10 *Diffie-Hellman idea*



15.3.1 Continued

Security of Diffie-Hellman

Discrete Logarithm Attack

Man-in-the-Middle Attack

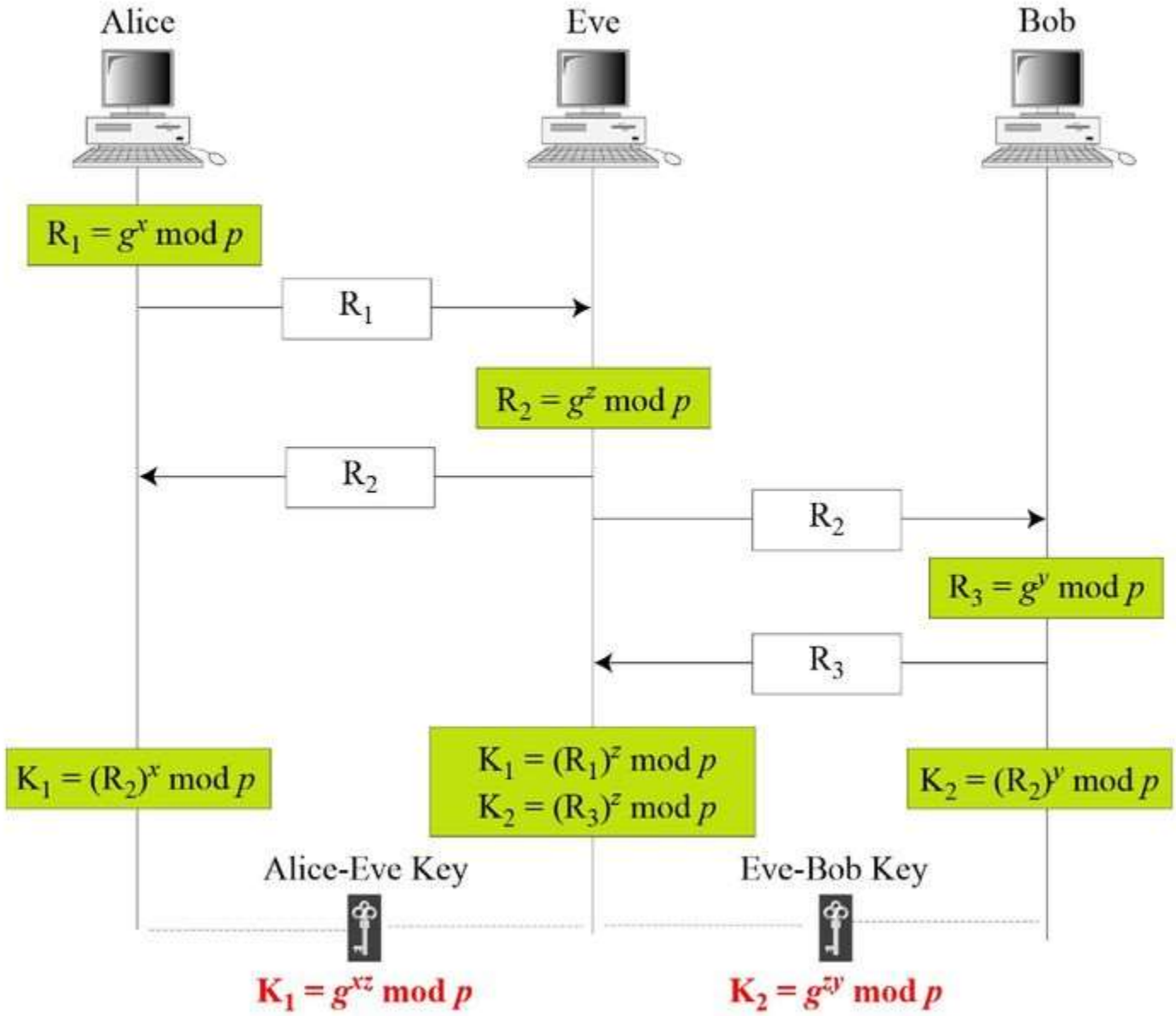
Discrete Logarithm Attacks

Eve can intercept R1 and R2. If she can find x from R1= g^x mod p and
y from R2=g^y mod p
Then she can easily calculate K = g^{xy} mod p .
To make Diffie-Hellman safe from this type of attack

1. The prime p must be very large (more than 300 decimal digits)
2. The prime p should be chosen such that p-1 has at-least one large prime factor
3. The generator must be chosen from <Zp*,*>
4. Bob and Alice must destroy x and y after they have calculated the key.
5. The said value of x and y must be used only once

15.3.1 Continued

Figure 15.11 *Man-in-the-middle attack*



Man-in-the-Middle Attack The protocol has another weakness. Eve does not have to find the value of x and y to attack the protocol. She can fool Alice and Bob by creating two keys: one between herself and Alice, and another between herself and Bob. Figure 15.11 shows the situation.

The following can happen:

1. Alice chooses x, calculates $R_1 = g^x \bmod p$, and sends R_1 to Bob.
2. Eve, the intruder, intercepts R_1. She chooses z, calculates $R_2 = g^z \bmod p$, and sends R_2 to both Alice and Bob.
3. Bob chooses y, calculates $R_3 = g^y \bmod p$, and sends R_3 to Alice. R_3 is intercepted by Eve and never reaches Alice.
4. Alice and Eve calculate $K_1 = g^{xz} \bmod p$, which becomes a shared key between Alice and Eve. Alice, however, thinks that it is a key shared between Bob and herself.
5. Eve and Bob calculate $K_2 = g^{zy} \bmod p$, which becomes a shared key between Eve and Bob. Bob, however, thinks that it is a key shared between Alice and himself.

15-4 PUBLIC-KEY DISTRIBUTION

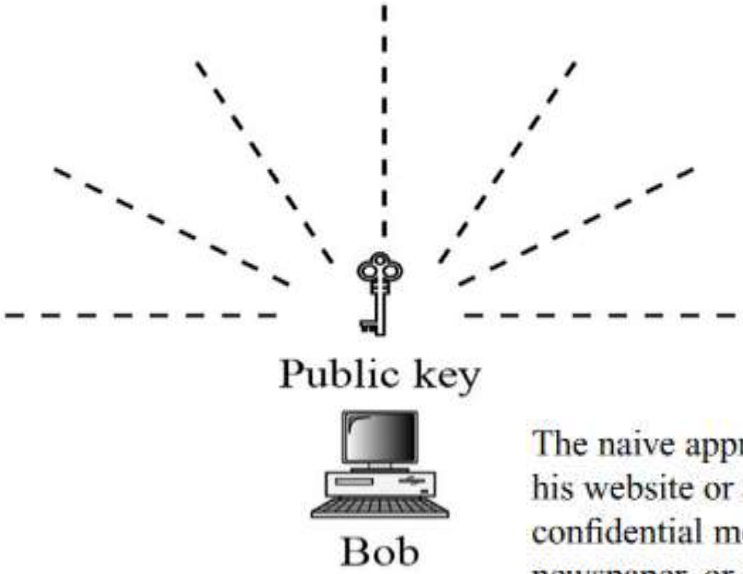
In asymmetric-key cryptography, people do not need to know a symmetric shared key; everyone shields a private key and advertises a public key.

Topics discussed in this section:

15.4.1 Public Announcement
15.4.2 Trusted Center
15.4.3 Controlled Trusted Center
15.4.4 Certification Authority
15.4.5 X.509
15.4.6 Public-Key Infrastructures (PKI)

Public Announcement

Figure 15.13 ***Announcing a public key***

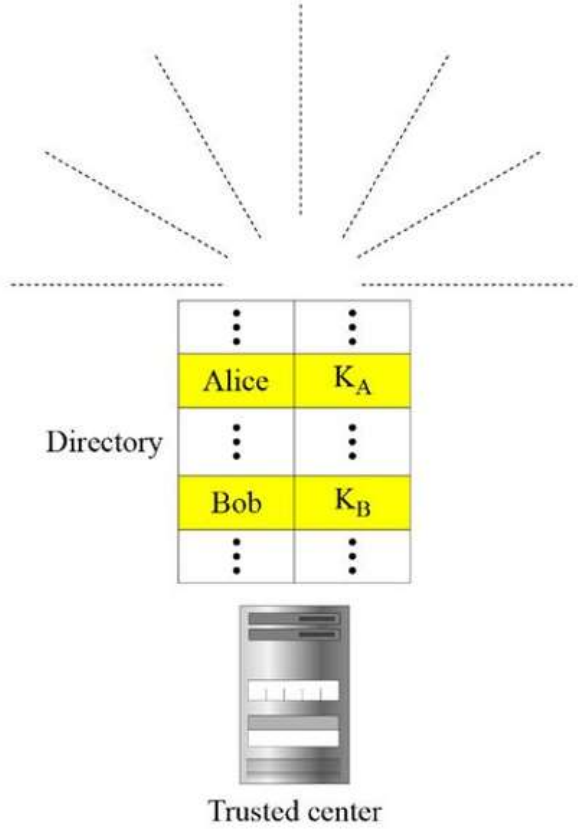


The naive approach is to announce public keys publicly. Bob can put his public key on his website or announce it in a local or national newspaper. When Alice needs to send a confidential message to Bob, she can obtain Bob's public key from his site or from the newspaper, or even send a message to ask for it. Figure 15.13 shows the situation.

This approach, however, is not secure; it is subject to forgery. For example, Eve could make such a public announcement. Before Bob can react, damage could be done. Eve can fool Alice into sending her a message that is intended for Bob. Eve could also sign a document with a corresponding forged private key and make everyone believe it was signed by Bob. The approach is also vulnerable if Alice directly requests Bob's public key. Eve can intercept Bob's response and substitute her own forged public key for Bob's public key.

Trusted Center

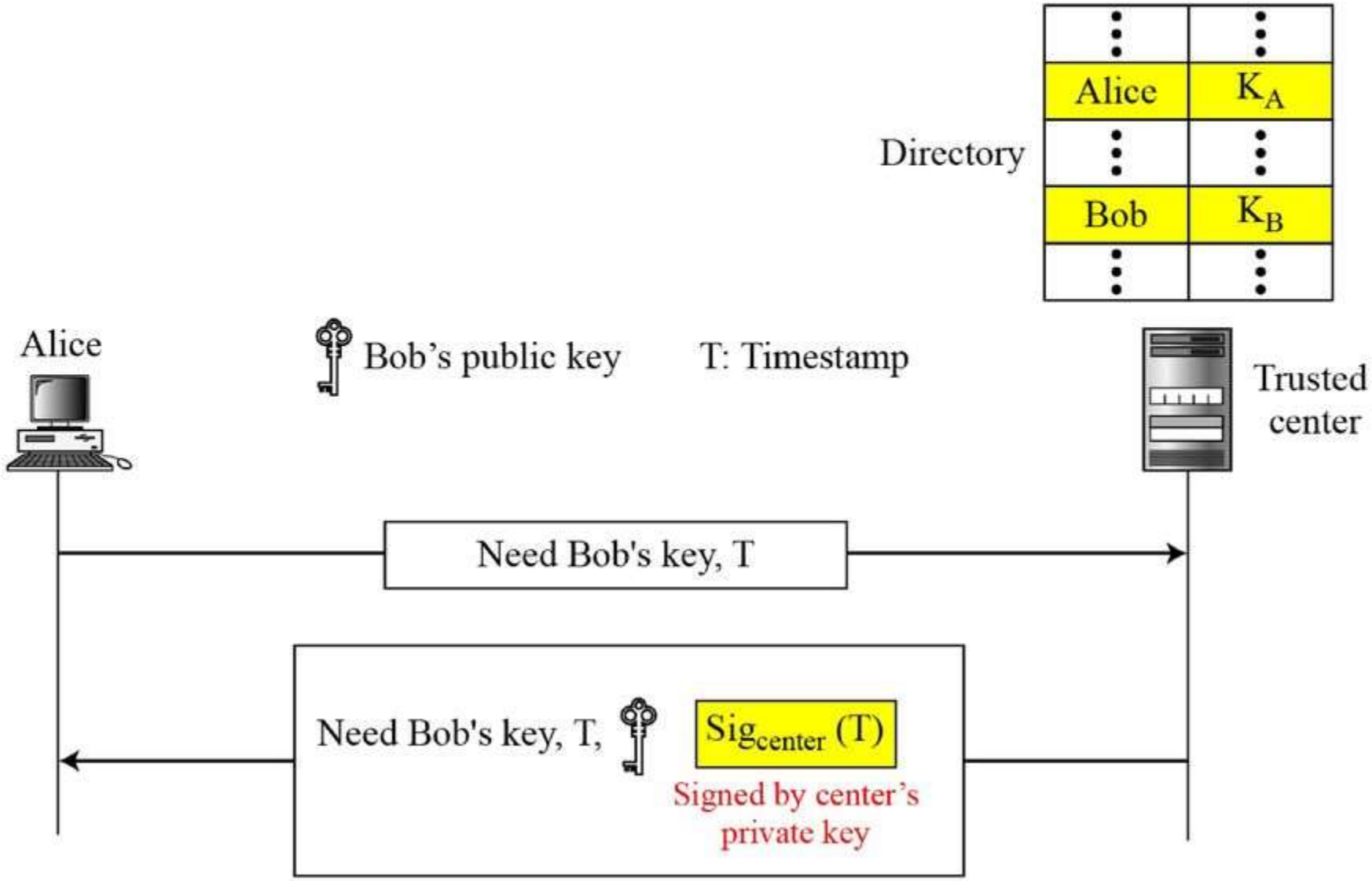
Figure 15.14 ***Trusted center***



A more secure approach is to have a trusted center retain a directory of public keys. The directory, like the one used in a telephone system, is dynamically updated. Each user can select a private and public key, keep the private key, and deliver the public key for insertion into the directory. The center requires that each user register in the center and prove his or her identity. The directory can be publicly advertised by the trusted center. The center can also respond to any inquiry about a public key. Figure 15.14 shows the concept.

15.4.3 Controlled Trusted Center

Figure 15.15 *Controlled trusted center*

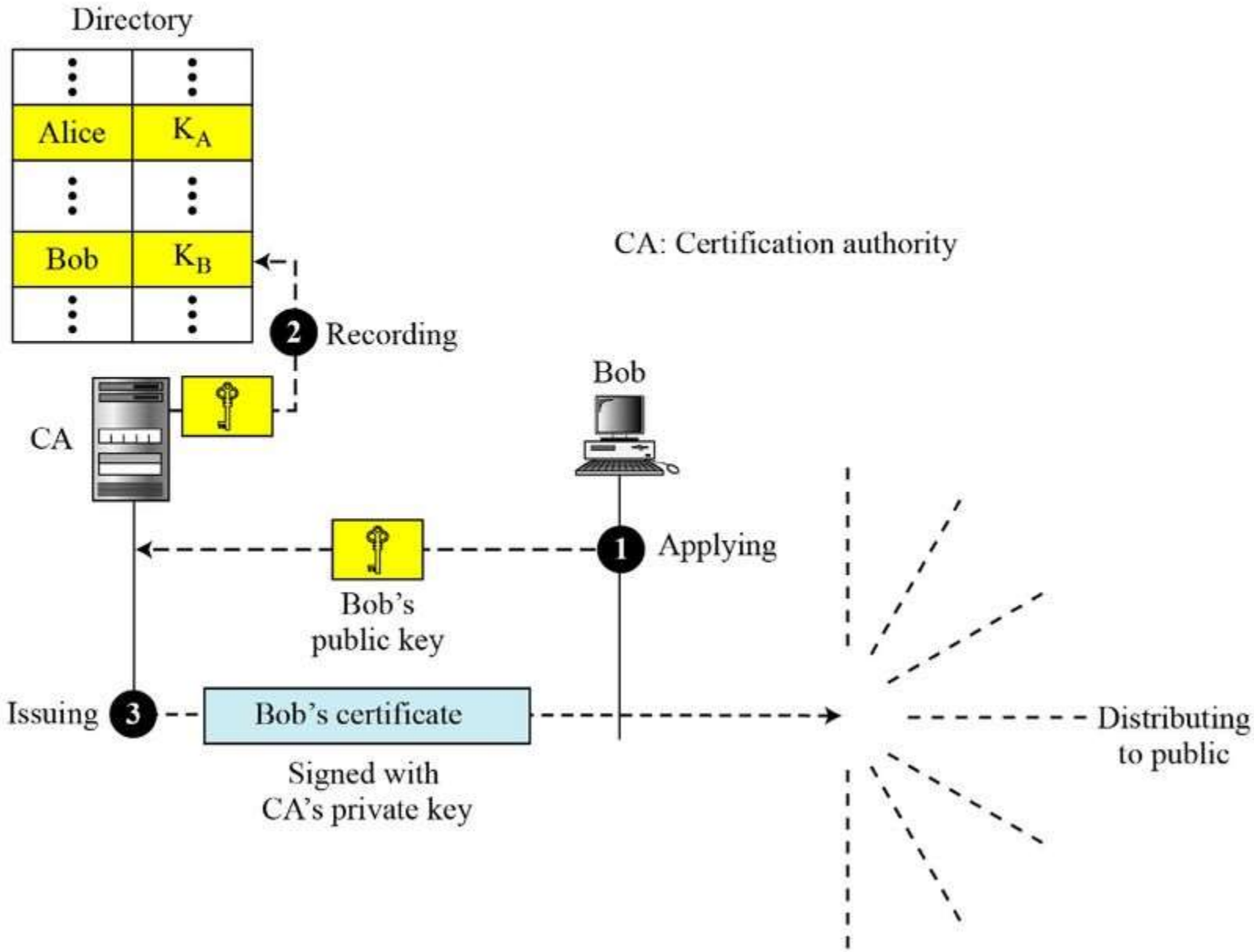


Controlled Trusted Center

A higher level of security can be achieved if there are added controls on the distribution of the public key. The public-key announcements can include a timestamp and be signed by an authority to prevent interception and modification of the response. If Alice needs to know Bob's public key, she can send a request to the center including Bob's name and a timestamp. The center responds with Bob's public key, the original request, and the timestamp signed with the private key of the center. Alice uses the public key of the center, known by all, to verify the timestamp. If the timestamp is verified, she extracts Bob's public key. Figure 15.15 shows one scenario.

15.4.4 Certification Authority

Figure 15.16 *Certification authority*

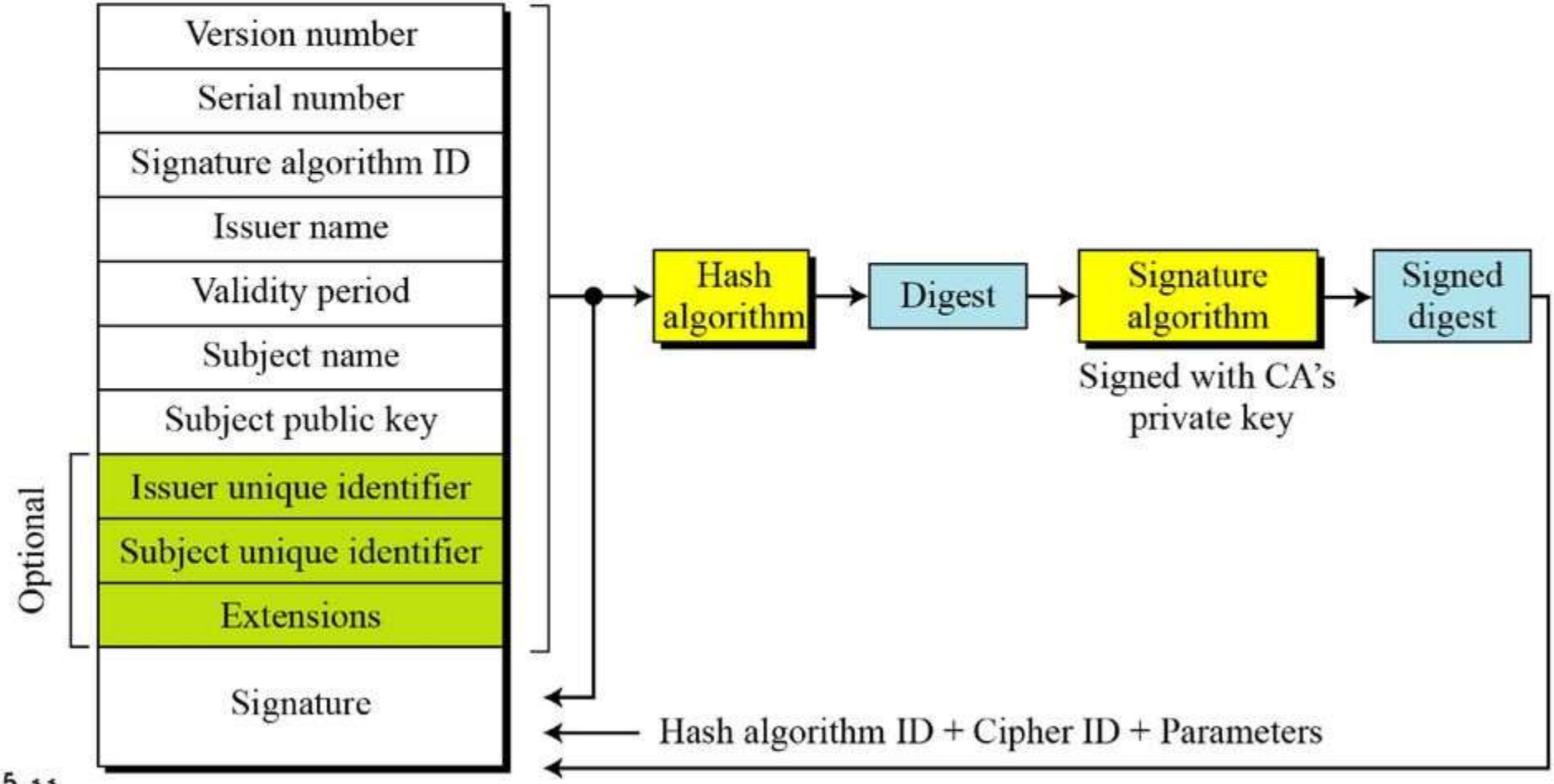


The previous approach can create a heavy load on the center if the number of requests is large. The alternative is to create **public-key certificates.** Bob wants two things; he wants people to know his public key, and he wants no one to accept a forged public key as his. Bob can go to a **certification authority (CA),** a federal or state organization that binds a public key to an entity and issues a certificate. The CA has a well-known public key itself that cannot be forged. The CA checks Bob's identification (using a picture ID along with other proof). It then asks for Bob's public key and writes it on the certificate. To prevent the certificate itself from being forged, the CA signs the certificate with its private key. Now Bob can upload the signed certificate. Anyone who wants Bob's public key downloads the signed certificate and uses the center's public key to extract Bob's public key. Figure 15.16 shows the concept.

15.4.5 X.509

Certificate

Figure 15.17 shows the format of a certificate.



A certificate has the following fields:

- **Version number.** This field defines the version of X.509 of the certificate. The version number started at 0; the current version (third version) is 2.
- **Serial number.** This field defines a number assigned to each certificate. The value is unique for each certificate issuer.
- **Signature algorithm ID.** This field identifies the algorithm used to sign the certificate. Any parameter that is needed for the signature is also defined in this field.
- **Issuer name.** This field identifies the certification authority that issued the certificate. The name is normally a hierarchy of strings that defines a country, a state, organization, department, and so on.
- **Validity Period.** This field defines the earliest time (not before) and the latest time (not after) the certificate is valid.
- **Subject name.** This field defines the entity to which the public key belongs. It is also a hierarchy of strings. Part of the field defines what is called the *common name,* which is the actual name of the beholder of the key.
- **Subject public key.** This field defines the owner's public key, the heart of the certificate. The field also defines the corresponding public-key algorithm (RSA, for example) and its parameters.
- **Issuer unique identifier.** This optional field allows two issuers to have the same *issuer* field value, if the *issuer unique identifiers* are different.
- **Subject unique identifier.** This optional field allows two different subjects to have the same *subject* field value, if the *subject unique identifiers* are different.
- **Extensions.** This optional field allows issuers to add more private information to the certificate.
- **Signature.** This field is made of three sections. The first section contains all other fields in the certificate. The second section contains the digest of the first section encrypted with the CA's public key. The third section contains the algorithm identifier used to create the second section.

15.4.5 Continued

Certificate Renewal

Each certificate has a period of validity. If there is no problem with the certificate, the CA issues a new certificate before the old one expires.

Certificate Renewal

In some cases a certificate must be revoked before its expiration.

Delta Revocation

To make revocation more efficient, the delta certificate revocation list (delta CRL) has been introduced.

Certificate Renewal

Each certificate has a period of validity. If there is no problem with the certificate, the CA issues a new certificate before the old one expires. The process is like the renewal of credit cards by a credit card company; the credit card holder normally receives a renewed credit card before the one expires.

Certificate Revocation

In some cases a certificate must be revoked before its expiration. Here are some examples:

a. The user's (subject's) private key (corresponding to the public key listed in the certificate) might have been comprised.
b. The CA is no longer willing to certify the user. For example, the user's certificate relates to an organization that she no longer works for.
c. The CA's private key, which can verify certificates, may have been compromised. In this case, the CA needs to revoke all unexpired certificates.

The revocation is done by periodically issuing a certificate revocation list (CRL). The list contains all revoked certificates that are not expired on the date the CRL is issued. When a user wants to use a certificate, she first needs to check the directory of the corresponding CA for the last certificate revocation list. Figure 15.18 shows the certificate revocation list.

A certificate revocation list has the following fields:

- ❑ **Signature algorithm ID.** This field is the same as the one in the certificate.
- ❑ **Issuer name.** This field is the same as the one in the certificate.
- ❑ **This update date.** This field defines when the list is released.
- ❑ **Next update date.** This field defines the next date when the new list will be released.
- ❑ **Revoked certificate.** This is a repeated list of all unexpired certificates that have been revoked. Each list contains two sections: user certificate serial number and revocation date.
- ❑ **Signature.** This field is the same as the one in the certificate list.

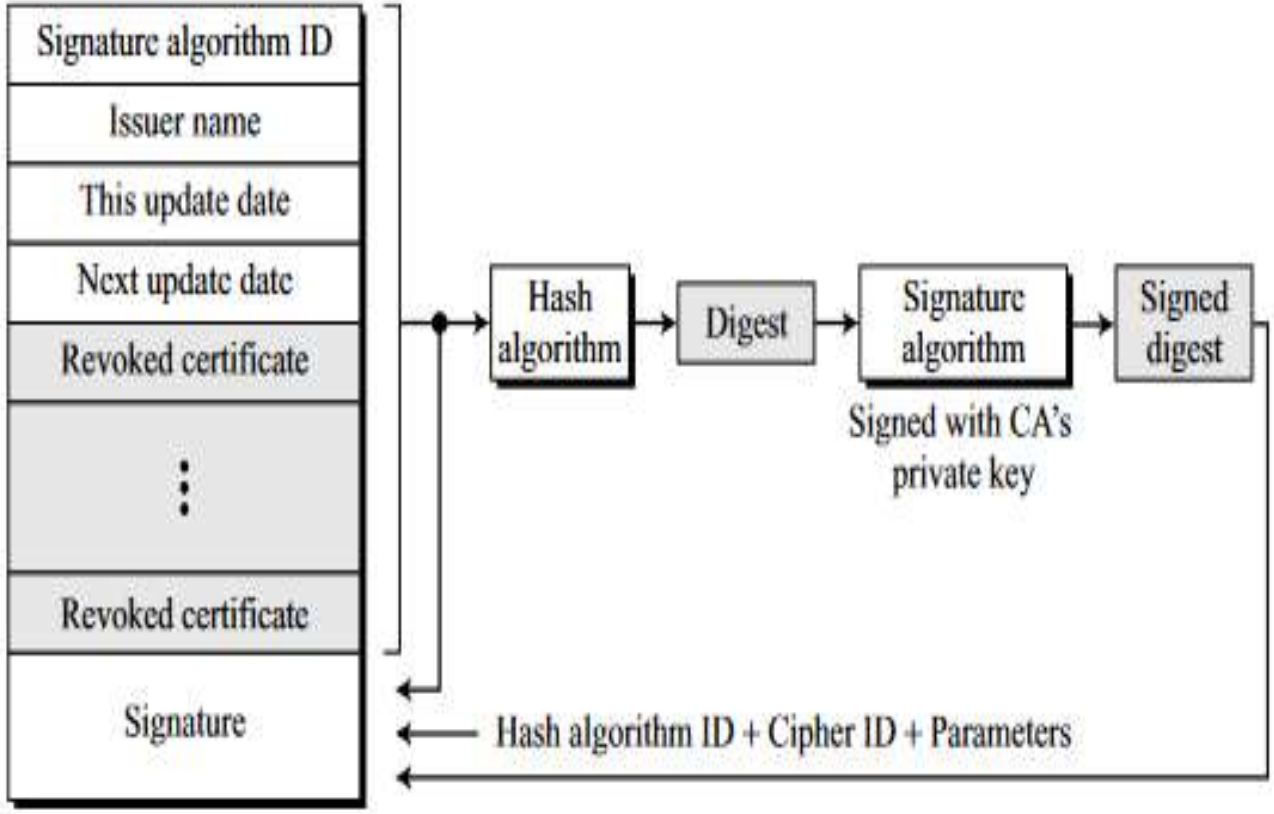



Figure 15.18 *Certificate revocation format*